

Subida de archivos al servidor

M8: IMPLEMENTACIÓN DE API BACKEND NODE EXPRESS

|AE3: IMPLEMENTAR LA FUNCIONALIDAD DE SUBIDA DE ARCHIVOS A UN SERVIDOR UTILIZANDO EXPRESS DE ACUERDO AL ENTORNO NODE.JS

Introducción

En el desarrollo backend moderno, muchas aplicaciones necesitan permitir a los usuarios **subir archivos** al servidor, ya sea para almacenar imágenes de perfil, documentos, o cualquier otro contenido digital. En el entorno de Node.js, una forma simple y efectiva de lograr esto es mediante el uso del paquete **express-fileupload**, un middleware que facilita la gestión de archivos en aplicaciones creadas con el framework Express.

Este manual te guiará paso a paso para implementar correctamente la funcionalidad de **upload de archivos** a un servidor Express, cubriendo desde la instalación y configuración inicial hasta el manejo de validaciones, atributos del archivo, renombramiento y eliminación.

Además, aplicaremos buenas prácticas para asegurar que la API sea clara, segura y funcional, cumpliendo con los requerimientos típicos de aplicaciones reales.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Reconocer los paquetes necesarios para implementar la subida de archivos en un servidor Express.
- Instalar y configurar el middleware **express-fileupload** en una aplicación Node.js.
- Crear un endpoint para subir archivos desde el cliente al servidor.
- Validar la existencia del archivo, su extensión y otros atributos clave.
- Modificar dinámicamente el nombre del archivo recibido.
- Responder al cliente con mensajes adecuados según el resultado de la operación.
- Acceder a propiedades del archivo subido y almacenarlo de forma organizada.
- Eliminar archivos del servidor cuando sea necesario.

¿Qué es **express-fileupload** y cómo se instala?

¿Qué es **express-fileupload**?

express-fileupload es un middleware para Express que permite **gestionar la subida de archivos desde el cliente al servidor** de forma simple y rápida. Al integrarlo en tu servidor, podés acceder a los archivos enviados en una petición HTTP tipo **POST** mediante **req.files**.

Este módulo convierte automáticamente los archivos cargados por el cliente en objetos accesibles, facilitando tareas como:

- Validar archivos
- Guardarlos en el servidor
- Acceder a su tipo, tamaño y nombre
- Renombrarlos o moverlos de ubicación



Instalación del paquete

Para usarlo, primero debés tener creado un proyecto Node.js. Si todavía no lo hiciste:

```
npm init -y
```

Luego instalá el paquete:

```
npm install express express-fileupload
```



Configuración básica del servidor

En tu archivo principal (**index.js** o **app.js**), agregá la configuración inicial:

```
const express = require('express');
const fileUpload = require('express-fileupload');
const app = express();

// Middleware para permitir archivos
app.use(fileUpload());

// Middleware para servir archivos estáticos (opcional)
app.use(express.static('public'));
```

```
app.listen(3000, () => {  
  console.log('Servidor escuchando en http://localhost:3000');  
});
```

Con esto, el servidor ya está listo para recibir archivos.

Subida básica de archivos al servidor

Ahora que ya tenemos instalado y configurado `express-fileupload`, vamos a crear un endpoint que permita **recibir un archivo desde el cliente** y guardarlo en el servidor.

Paso 1: Crear un formulario HTML (opcional para pruebas)

Podés usar este HTML básico para probar desde el navegador:

```
<!-- Guardalo como public/index.html -->  
<form ref="uploadForm"  
  id="uploadForm"  
  action="/upload"  
  method="POST"  
  encType="multipart/form-data">  
  <input type="file" name="archivo" />  
  <input type="submit" value="Subir archivo" />  
</form>
```

Paso 2: Crear el endpoint `/upload` en Express

```
app.post('/upload', (req, res) => {  
  // Verificar si se envió un archivo  
  if (!req.files || Object.keys(req.files).length === 0) {  
    return res.status(400).json({ mensaje: 'No se subió ningún archivo' });  
  }  
  
  // Obtener el archivo (nombre del campo: 'archivo')  
  const archivo = req.files.archivo;  
  
  // Ruta donde se guardará el archivo  
  const ruta = `${__dirname}/uploads/${archivo.name}`;  
  
  // Mover el archivo a la carpeta "uploads"
```

```

archivo.mv(ruta, (err) => {
  if (err) {
    return res.status(500).json({ mensaje: 'Error al guardar el archivo', error: err });
  }

  res.status(200).json({ mensaje: 'Archivo subido con éxito', nombre: archivo.name });
});
});

```

Recomendaciones

- Creá una carpeta llamada **uploads** en la raíz del proyecto para almacenar los archivos.
- Asegurate de agregar **express-fileupload** al middleware con **app.use(fileUpload());**
- El nombre del campo del formulario HTML (**name="archivo"**) debe coincidir con **req.files.archivo**.

Resultado esperado

Cuando enviás un archivo desde el formulario o Postman, el servidor lo guarda en la carpeta **/uploads** y responde con un mensaje de éxito:

```

{
  "mensaje": "Archivo subido con éxito",
  "nombre": "documento.pdf"
}

```

Validaciones (existencia, tipo de archivo y nombre)

Cuando permitís subir archivos a un servidor, es fundamental validar lo que recibís. Esto evita errores, sobrescrituras o incluso amenazas de seguridad.

Validación 1: ¿Se envió algún archivo?

Esto ya lo hicimos en el tema anterior:

```

if (!req.files || Object.keys(req.files).length === 0) {
  return res.status(400).json({ mensaje: 'No se subió ningún archivo' });
}

```

✓ Validación 2: Tipo de archivo (extensión)

Podés permitir solo ciertos tipos de archivo, por ejemplo imágenes (.png, .jpg, .jpeg):

```
const archivo = req.files.archivo;
const nombre = archivo.name;
const extension = nombre.split('.').pop().toLowerCase();

// Extensiones permitidas
const extensionesValidas = ['png', 'jpg', 'jpeg'];

if (!extensionesValidas.includes(extension)) {
  return res.status(400).json({
    mensaje: 'Extensión no permitida',
    extensionesPermitidas: extensionesValidas
  });
}
```

✓ Validación 3: Cambiar el nombre del archivo

Para evitar conflictos y garantizar unicidad, podés renombrarlo con un identificador único:

```
const nombreFinal = `${Date.now()}.${extension}`;
// ej: 1708107621493.jpg
const ruta = `${__dirname}/uploads/${nombreFinal}`;

archivo.mv(ruta, (err) => {
  if (err) {
    return res.status(500).json({ mensaje: 'Error al guardar el archivo' });
  }
  res.status(200).json({ mensaje: 'Archivo subido', nombre: nombreFinal });
});
```

🧠 Bonus: Validar el tamaño del archivo (opcional)

Podés limitar el tamaño máximo permitido desde la configuración:

```
app.use(fileUpload({
  limits: { fileSize: 2 * 1024 * 1024 } // 2MB
}));
```

Si se supera ese límite, `express-fileupload` responde automáticamente con error `413 Payload Too Large`.

Responder al cliente y acceder a atributos del archivo

Una vez que el servidor recibe un archivo, es muy útil **informar al cliente** sobre el estado de la operación, y también devolver datos del archivo subido, como nombre, tipo, tamaño, etc.

¿Qué atributos puedo obtener del archivo?

Cuando subís un archivo usando `express-fileupload`, se crea un objeto en `req.files` con la siguiente estructura:

```
{
  name: 'foto.jpg',
  data: <Buffer>,
  size: 24832,
  encoding: '7bit',
  tempFilePath: '',
  truncated: false,
  mimetype: 'image/jpeg',
  md5: 'd41d8cd98f00b204e9800998ecf8427e',
  mv: [Function: mv]
}
```

✓ Ejemplo de respuesta detallada al cliente

```
app.post('/upload', (req, res) => {
  if (!req.files || !req.files.archivo) {
    return res.status(400).json({ mensaje: 'No se subió ningún archivo' });
  }

  const archivo = req.files.archivo;
  const extension = archivo.name.split('.').pop().toLowerCase();
  const nombreFinal = `${Date.now()}.${extension}`;
  const ruta = `${__dirname}/uploads/${nombreFinal}`;

  archivo.mv(ruta, (err) => {
    if (err) {
      return res.status(500).json({ mensaje: 'Error al guardar el archivo' });
    }
  });
});
```

```

}

res.status(200).json({
  mensaje: 'Archivo subido con éxito',
  nombreOriginal: archivo.name,
  nombreFinal: nombreFinal,
  tipo: archivo.mimetype,
  tamaño: `${(archivo.size / 1024).toFixed(2)} KB`
});
});
});

```

🧠 ¿Por qué esto es importante?

- Le das al usuario feedback claro.
- Podés usar estos datos en futuras validaciones (por ejemplo, limitar solo imágenes pequeñas).
- Es buena práctica para debugging y registro de actividad.

Eliminación de archivos del servidor

En muchos casos, además de subir archivos, vas a necesitar **eliminarlos** del servidor, ya sea porque:

- El usuario borra o reemplaza su archivo
- Querés liberar espacio
- O simplemente porque el archivo ya no es necesario

¿Cómo se eliminan archivos en Node.js?

Para eliminar archivos usamos el módulo nativo **fs** (file system) de Node.js.

```
const fs = require('fs');
```

🔧 Ejemplo básico de eliminación

```

app.delete('/archivo/:nombre', (req, res) => {
  const nombreArchivo = req.params.nombre;
  const ruta = `${__dirname}/uploads/${nombreArchivo}`;

  // Verificar si existe
  fs.access(ruta, fs.constants.F_OK, (err) => {

```

```
if (err) {  
  return res.status(404).json({ mensaje: 'Archivo no encontrado' });  
}  
  
// Eliminar  
fs.unlink(ruta, (err) => {  
  if (err) {  
    return res.status(500).json({ mensaje: 'No se pudo eliminar el archivo' });  
  }  
  
  res.status(200).json({ mensaje: 'Archivo eliminado exitosamente' });  
});  
});  
});
```

✓ Resultado esperado

DELETE /archivo/foto_123.jpg

→ Respuesta:

```
{  
  "mensaje": "Archivo eliminado exitosamente"  
}
```

🧠 Consejo extra

- Siempre validá que el archivo exista antes de eliminarlo.
- Nunca permitas eliminar cualquier archivo arbitrario sin control: podrías exponer el servidor a riesgos. Filtrá o controlá los nombres de los archivos permitidos.

Cierre

A lo largo de este manual aprendiste a implementar una funcionalidad clave en muchas aplicaciones modernas: **la subida y gestión de archivos en un servidor Express**.

Exploraste cómo instalar y configurar el middleware `express-fileupload`, cómo crear rutas para recibir archivos, cómo realizar validaciones de seguridad y formato, y cómo responder adecuadamente al cliente. Además, incorporaste la capacidad de **eliminar archivos** del servidor, completando el ciclo básico de gestión de archivos.

Dominar esta funcionalidad te permitirá desarrollar aplicaciones más interactivas y robustas, como sistemas de gestión de usuarios, galerías, portales de carga de documentos o plataformas de contenidos.

Referencias

- express-fileupload. (s.f.). NPM.
<https://www.npmjs.com/package/express-fileupload>
- Express.js. (s.f.). Documentación oficial de Express. <https://expressjs.com/es/>
- Node.js. (s.f.). Módulo fs (file system). <https://nodejs.org/api/fs.html>
- Mozilla. (s.f.). Códigos de estado HTTP. MDN Web Docs.
<https://developer.mozilla.org/es/docs/Web/HTTP/Status>

¡Muchas gracias!

Nos vemos en la próxima lección

